

TRAKEY

App Building Blueprint & Learning Guide

From first prototype → persistent productivity app → stable v2.1 release

Purpose: A practical study/reference document for learning how an Android app was planned, coded, debugged, built, installed, tested, and released.

Stable milestone: Trakey v2.1.0 • applicationId com.trakey.app •
versionCode 21

1. What We Built

Trakey is a productivity hub designed around a simple idea: keep everyday student/general productivity tools in one lightweight app. The final v2.1 build contains a Home dashboard, Goals, Focus timer, Notes, and Tools including Calculator, Stopwatch, Recorder, Currency, Learn, and an in-app Trakey AI interface.

Important architecture lesson: the final project is intentionally simple. Most UI and app logic lives in one large `MainActivity.kt`. This is useful for learning and prototyping, but a larger production app would normally split features into multiple files, view models, repositories, and backend layers.

The v2.1 AI screen is an in-app assistant UI with local keyword-based replies. It is **not** a full generative AI backend. The planned next major release is a secure backend/API connection so real AI can work inside the app without exposing an API secret in the APK.

2. Technology Stack

Layer	Technology used	What it does
Language	Kotlin	Main Android programming language
UI	Jetpack Compose + Material 3	Declarative modern Android interface
Build	Gradle + Android Gradle Plugin	Compiles and packages the app
IDE	Android Studio	Project editing, SDK management, Gradle sync/build
Target	Android SDK 35	Compile/target platform
Minimum	Android API 26	Oldest supported Android version in this project
Storage	SharedPreferences + JSON	Local persistence for goals, notes, focus state
Audio	MediaRecorder + MediaPlayer	Recording and playback
Network	HTTP request to Frankfurter API	Currency conversion
Device testing	ADB	Install/debug on a physical Android phone

3. How We Started

The first prototype was deliberately small: a dark dashboard, daily target list, Focus timer, Notes, and Tools placeholders. The first project README explicitly treated persistence, calculator, stopwatch, recorder, currency, learning links, and AI as later work.

```
Project opening flow Android Studio → New/Open project → Gradle sync → Build APK → Install/test
Typical terminal build ./gradlew :app:assembleDebug Typical device install adb install -r
app/build/outputs/apk/debug/app-debug.apk
```

4. Version-by-Version Growth

v0.1 — Foundation: Home, Goals/targets, Focus, Notes, Tools placeholders. The goal was to prove the basic app structure and navigation.

v0.2 — Persistence + tools: Notes were made persistent using SharedPreferences. Goals were expanded to JSON persistence with add/edit/delete/complete behavior and daily completion reset. Focus persistence was added. Calculator, Stopwatch with laps, and other utility logic were introduced.

v0.3 — Connectivity + media: Recorder, Currency conversion, Learn links, and an AI entry point were added. Currency used an external HTTP API; Learn used external resource links; AI initially opened ChatGPT.

v0.3.1 — Recorder repair: Recorder playback was repaired using MediaPlayer with asynchronous preparation, error handling, and cleanup.

v2.0 — Major redesign: The project moved from prototype styling to a more complete app experience: premium dark UI, splash screen, key-inspired branding, cleaner hierarchy, animated transitions, dashboard, tools, local persistence, and an AI entry point.

v2.1 — Polish/stable release: Custom launcher icon, microphone permission flow, lighter transitions, Home footer, compact AI button, proper Android Back handling, and an in-app AI chat screen were added. This became the stable release milestone.

5. Final Project File Map

Path	Purpose
settings.gradle.kts	Gradle project settings and included modules
build.gradle.kts	Top-level Gradle configuration/plugins
gradle.properties	Gradle/project properties
gradlew / gradlew.bat	Gradle wrapper commands for Linux/Windows
gradle/wrapper/*	Gradle wrapper configuration/JAR
app/build.gradle.kts	Android app module, SDK versions, dependencies, Kotlin/Java targets
app/src/main/AndroidManifest.xml	App permissions, icon, theme, launcher activity
app/src/main/java/com/trakey/app/MainActivity.kt	MainActivity plus Compose UI, state, persistence, tools, recorder, currency, AI UI
app/src/main/res/drawable/ic_trakey.xml	Vector Trakey launcher/app icon
app/src/main/res/values/styles.xml	Base Android theme/status/navigation bar styling
README.md	Basic project/build documentation

6. Understanding the Main Code Structure

MainActivity starts the Compose UI. TrakeyApp owns high-level navigation state such as the selected bottom tab, selected tool, and goals. Individual composable functions render screens and receive callbacks for actions.

```
class MainActivity : ComponentActivity() { override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState) setContent { TrakeyApp() } } } @Composable fun TrakeyApp() { //  
    high-level state // theme // splash // back handling // bottom navigation // screen/tool selection }
```

Learning principle: Compose is declarative. Instead of manually finding and changing views, you describe what the UI should look like for the current state. When state changes, Compose recomposes the affected UI.

7. Navigation Without a Complex Navigation Graph

The stable build uses simple state variables: a Tab enum for the five main areas and a Tool enum for tool screens. This keeps the prototype easy to understand.

```
enum class Tool { CALCULATOR, STOPWATCH, RECORDER, CURRENCY, LEARN, AI } enum class Tab(val label:  
String, val icon: ImageVector) { HOME("Home", Icons.Default.Home), GOALS("Goals",  
Icons.Default.CheckCircle), FOCUS("Focus", Icons.Default.Timer), NOTES("Notes",  
Icons.Default.EditNote), TOOLS("Tools", Icons.Default.Build) }
```

For a larger app, Navigation Compose routes or a more formal navigation architecture may be preferable. For learning, the state-based approach makes it obvious how screens are selected.

8. Local Persistence: The Core Idea

Goals and notes need to survive app restarts. SharedPreferences is a simple key/value storage mechanism. For lists of custom objects, the project converts objects to JSON strings using JSONArray/JSONObject and stores the resulting text.

```
private const val NOTES_PREFS = "trakey_notes_v2" private const val NOTES_KEY = "notes" val prefs =  
context.getSharedPreferences( NOTES_PREFS, Context.MODE_PRIVATE ) prefs.edit() .putString(NOTES_KEY,  
jsonString) .apply()
```

Study upgrade: For a larger production app, learn Room/SQLite and repository patterns.

SharedPreferences is excellent for small settings and simple prototype data, but it is not a replacement for a full database.

9. Focus Timer Lesson

The Focus timer stores remaining time, end time, and whether it is running. The important concept is that a timer should not depend only on a counter in memory. Persisting an end timestamp lets the app reconstruct the remaining time after recreation.

```
remaining = maxOf(0L, endTime - System.currentTimeMillis()) // Concept: // endTime = currentTime +  
duration // remaining = endTime - currentTime
```

10. Calculator Lesson

The calculator was implemented with a custom expression parser rather than evaluating arbitrary code. This is a useful security lesson: never solve calculator input by executing user text as program code.

11. Stopwatch Lesson

A stopwatch needs elapsed-time measurement and lap storage. Android's SystemClock.elapsedRealtime() is appropriate for elapsed duration because it is based on a monotonic clock rather than wall-clock time.

12. Recorder: Permissions + MediaRecorder

Recording audio requires the RECORD_AUDIO permission. Modern Android apps should request dangerous permissions at runtime and only start recording after permission is granted.

```
<uses-permission android:name="android.permission.RECORD_AUDIO" /> // Runtime idea: if permission is granted: start recording else: request permission start only after a successful result
```

The recorder writes to the app's external-files recording directory and uses MediaRecorder with an MPEG-4 container and AAC audio. Playback uses MediaPlayer.

13. Recorder Bug We Faced

Problem: the first recorder implementation created recordings, but playback did not work correctly. This led to the v0.3.1 recorder fix.

Fix: use MediaPlayer with proper asynchronous preparation, error handling, and release/cleanup. Media resources must be released when playback finishes or fails.

Second v2.1 problem: the permission callback referenced `startRecordingNow()` where the local function was not visible/resolved. The function was moved before the callback so Kotlin could resolve it.

14. Currency API Lesson

The Currency tool demonstrates the difference between offline and online features. The app requests exchange-rate data from the Frankfurter service. Network operations should not block the UI thread; the Compose implementation uses coroutine-based state to represent loading, result, and error states.

General rule: every network feature should have loading, success, and failure states. Assume the network can be slow or unavailable.

15. Learn Links and External Intents

The Learn area can open external educational resources. Android Intent/URI handling is the standard pattern when an app needs to hand a task to a browser or another installed application.

16. AI: What We Have Now vs. What Comes Next

The current v2.1 AI screen is fully inside Trakey, but its responses are local keyword-based replies. That makes it useful for demonstrating chat UI, state, message lists, and interaction flow without needing a server.

```
data class Message( val text: String, val fromUser: Boolean ) // v2.1 flow user types message → add
user message to list → local reply() checks keywords → add assistant message → display both in chat
```

Next major release: a real AI backend. The Android app should send a request to a backend that authenticates the user/request and calls the AI provider. Never hard-code a private provider API key into the APK. Anything shipped inside an APK can potentially be extracted.

17. The Exact Build/Environment Issue We Faced

At one point Gradle could not find the Android SDK because the project did not have the correct local.properties path. Android Studio showed the SDK at `/home/ryadav/Android/Sdk`. The project was fixed by creating local.properties with the SDK path.

```
echo "sdk.dir=/home/ryadav/Android/Sdk" > local.properties ./gradlew :app:assembleDebug
```

Lesson: local.properties is machine-specific. Do not commit a personal SDK path into a shared project. Each developer should point local.properties to their own Android SDK.

18. v2.1 Compilation Errors and Fixes

Error 1 — NoteEditor:

```
No value passed for parameter 'onSave'. Argument type mismatch: Function1<...> but Function0<...>
expected. Cannot infer type for this parameter.
```

Cause: a trailing lambda was being interpreted as the wrong parameter because onDelete had a default/optional position. Kotlin could not infer that the lambda belonged to onSave.

Fix: use named parameters and explicitly name the text parameter.

```
NoteEditor( note = null, onDismiss = { creating = false }, onSave = { text -> val item = NoteItem(
System.currentTimeMillis(), text, System.currentTimeMillis() ) notes = listOf(item) + notes
saveNotes(context, notes) creating = false } )
```

Error 2 — Recorder:

```
Unresolved reference 'startRecordingNow'
```

Cause: the permission result callback tried to call a local helper function that was declared outside the callback's visible scope/order.

Fix: move/define startRecordingNow() before the permission launcher callback and let the callback call it after permission is granted.

Error 3 — MediaRecorder error listener:

```
setOnErrorListener { it.release(); ... true }
```

The listener receives three parameters. The original lambda treated the wrong object as the MediaPlayer/MediaRecorder instance.

Fix pattern:

```
setOnErrorListener { mp, _, _ -> mp.release() // update UI error state true }
```

19. UI/UX Problems We Deliberately Fixed

Problem	Fix / lesson
Generic Android launcher icon	Added ic_trakey.xml and declared android:icon/roundIcon in the manifest; bumped versionCode for the release.
Tools/screen animations felt laggy	Replaced heavier slide/fade transition with shorter lightweight fade-in/fade-out timing.
Home AI button took too much space	Changed extended button into a compact circular FloatingActionButton with an AI icon.
Footer was missing at the bottom	Added Trakey ♥ and Powered by Maniaxe ! near the bottom content area.
Back button exited app	Added Compose BackHandler: tool → tab → Home → exit.
Recorder button/permission flow	Added runtime RECORD_AUDIO permission request and start-after-grant behavior.

20. Warnings Are Not Always Errors

During the successful build, Gradle reported: *Deprecated Gradle features were used in this build, making it incompatible with Gradle 10.* This is a warning, not the reason the current build failed. A build can be successful while still reporting warnings.

Learn to distinguish: **error** = compilation/build cannot complete; **warning** = something may need attention but build can continue; **runtime bug** = app builds but behaves incorrectly on the device.

21. Debugging Workflow We Learned

```
1. Build normally. 2. If it fails, read the FIRST useful compiler error. 3. Ignore the long Gradle
summary until the actual error is known. 4. Narrow Kotlin compiler output: ./gradlew
:app:compileDebugKotlin 2>&1 | grep -E 'e: |error:' 5. Fix one root cause. 6. Build again. 7. Install
on the real phone. 8. Test the changed feature. 9. Only then move to the next issue.
```

This workflow is more reliable than changing many files at once. The final v2.1 success came from reading exact compiler messages and correcting the source rather than guessing.

22. Physical Device Testing with ADB

ADB (Android Debug Bridge) is extremely useful for learning because it lets you install and inspect an app without manually copying APKs.

```
adb devices -l adb install -r app/build/outputs/apk/debug/app-debug.apk # expected successful
installation Success
```

If an APK behaves differently from the source you think you built, first confirm the APK was rebuilt and reinstall it. When launcher icons appear stale, uninstalling the old app or bumping versionCode can help the launcher refresh its cached icon.

23. A Beginner's Android Project Checklist

1. Define the app's purpose and smallest useful feature set.
2. Choose Kotlin + Jetpack Compose for a modern Android starting point.
3. Create a clean Gradle project and confirm it builds before adding features.
4. Build one screen at a time and test frequently.
5. Use state intentionally; understand recomposition.
6. Persist only what must survive app restarts.
7. Request dangerous permissions at runtime.
8. Handle loading, success, empty, and error states.
9. Never block the UI thread with network or long-running work.

10. Release media resources.
11. Do not put secret API keys in an Android APK.
12. Use ADB and a real device for testing.
13. Read compiler errors literally before guessing.
14. Separate warnings from actual build failures.
15. Freeze stable releases before starting risky experiments.

24. Suggested Architecture for Your Next Project

Trakey v2.1 is a great learning prototype, but your next serious project can improve the structure without changing the concepts.

```
app/ ui/ screens/ components/ theme/ data/ local/ remote/ repository/ domain/ models/ usecases/  
viewModel/ MainActivity.kt backend/ API routes authentication AI service rate limits logging  
secrets/config
```

A useful progression is: prototype first → understand the feature → refactor into architecture → add backend/security → test → release. Don't start with a huge architecture if you are still learning the basics.

25. What a Real AI Backend Would Teach You

For the next Trakey generation, the learning path can be:

1) Android chat UI → 2) HTTP/JSON basics → 3) backend endpoint → 4) secure environment variables → 5) AI provider request → 6) response parsing → 7) loading/error states → 8) authentication/rate limiting → 9) usage limits/cost control → 10) production monitoring.

The most important security rule is simple: the Android app is a client, not a safe place for server secrets. Keep private credentials on the backend.

26. Final Release Lessons

The biggest lesson from Trakey was not a particular Kotlin function. It was the development cycle: start small, build, test, encounter errors, read the actual compiler message, fix the root cause, rebuild, install, and repeat.

The stable v2.1 release demonstrates that a beginner can build a useful Android app without starting with a giant codebase. The next skill level is learning how to turn that working prototype into a maintainable architecture and then safely connect it to real online services.

27. Quick Reference Commands

```
# Check project pwd ls # Set SDK path for this project echo "sdk.dir=/home/ryadav/Android/Sdk" >  
local.properties # Build debug APK ./gradlew :app:assembleDebug # Compile Kotlin only ./gradlew  
:app:compileDebugKotlin # Show Kotlin compiler errors ./gradlew :app:compileDebugKotlin 2>&1 | grep  
-E 'e: |error:' # Install APK adb install -r app/build/outputs/apk/debug/app-debug.apk # Check  
connected devices adb devices -l
```

28. Trakey v2.1 Release Snapshot

Item	Final value
App name	Trakey
Version	2.1.0
versionCode	21
Package	com.trakey.app
Language	Kotlin
UI	Jetpack Compose + Material 3
compileSdk / targetSdk	35 / 35
minSdk	26
Java/Kotlin JVM target	17
Stable status	Released / frozen until bug or next major update
Next major milestone	Secure AI backend for real in-app generative AI

Keep this PDF as a study blueprint, not just documentation. When starting your next app, try to reproduce the process yourself before looking back at the solution.

Trakey ■ — Key to Success